Test Driven Development with OpenAI Codex



Akmal Khan

Senior Full Stack AI Architect

akmalkhaniub.github.io

Author Audition

Overview



Maliha's ticket: $100 should cost $90

BOOKSTORE CHECKOUT

Subtotal	$100.00
Discount	$0.00
Total	**$100.00**

One character, > vs >=, overcharges this customer.

Should be $90.00
exactly $100 qualifies

Specify with examples → **Delegate to Codex** → **Verify the result**

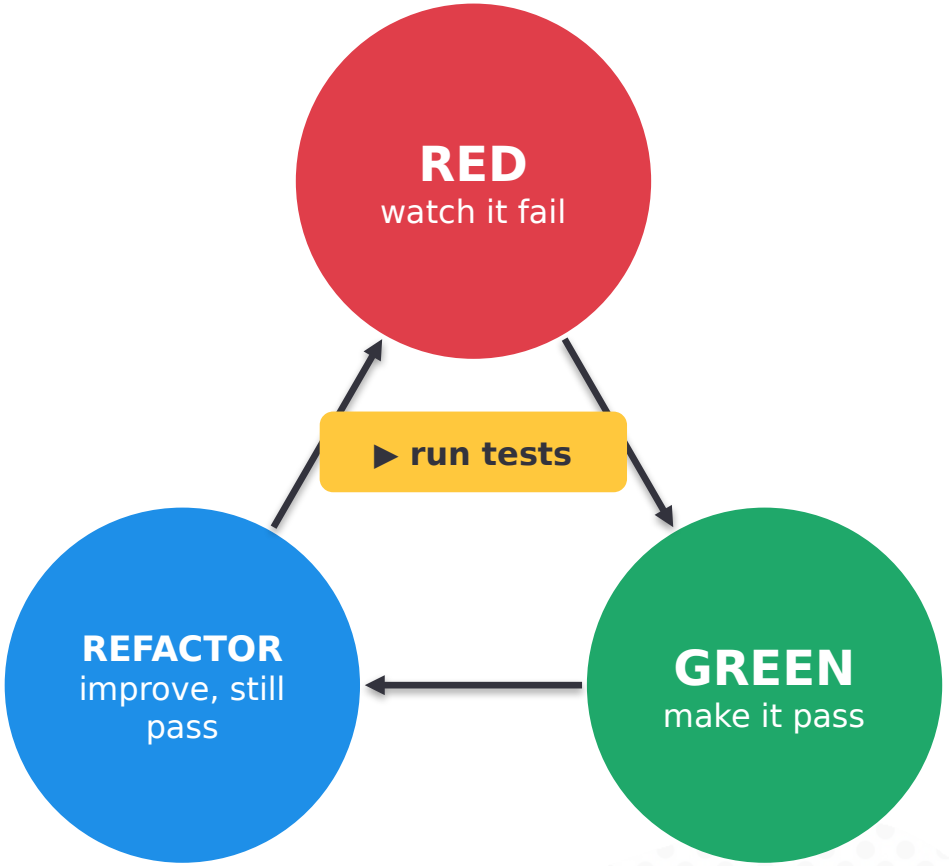
Your takeaway: a workflow where you, not the agent, decide what "correct" means.

Orders of $100 or more get 10% off

Order amount		Final total	
1¢ below $100: no discount	$99.99	$99.99	
exactly $100: discount	$100.00	$90.00	← >= includes $100
1¢ above $100: discount	$100.01	$90.01	
rounding case	$100.05	$90.05	← 90.045 rounds half up

Author Audition

The TDD cycle



Never saw it fail? Then it may be testing nothing.

Author Audition

The prompt contract

```
# Codex prompt
Orders of $100+ get 10% off.
Round with ROUND_HALF_UP.
Modify only discount_service.py.
Do not change the tests.
Run: python -m unittest -v
     test_discounts
On conflict: stop and explain.
```

RULE
what "correct" means

SCOPE
where Codex may edit

PROOF
how we check it

A request, not a lock. We still review the diff.

Demo



What we'll do in Maliha's project

1. **RED**
   Run 8 tests, see 8 expected errors
2. **PROMPT**
   Give Codex the contract
3. **REVIEW**
   Read the diff: >= and rounding
4. **GREEN**
   Same 8 tests now pass
5. **REFACTOR?**
   Improve only if useful; still green

Your next TDD cycle

An order of -$5.00 arrives.

What should calculate_total return?



Then run the whole suite again.

Summary



Examples turn a rule into a check

Scope keeps the diff reviewable

The same tests guard the refactor

Green proves only what you tested

Author Audition